ECG Signal Processing for Anaesthesia Monitoring: A Rule-Based Classification Approach

A Preprint

Sarath Adukkadukkam
Faculty of Electrical Engineering, Media and Computer Science
OTH Amberg-Weiden
s.adukkadukkam@oth-aw.de

January 18, 2026

Abstract

This report walks through a simulation-based approach to ECG signal processing for anaesthesia monitoring. I have generated synthetic ECG signals that mimic awake and sleep states, filtered them according to IEC 60601-2-27 standards using a band-pass filter, detected R-peaks, and extracted heart rate (HR) and heart rate variability (HRV) features to classify what state the patient is in. To test how robust the system is, I added some realistic operating-room challenges, such as electrocautery interference and electrode lead-off events. The processing chain nails 100% classification accuracy on our synthetic data and effectively crushes 100 Hz interference with 31.4 dB attenuation. This work lays the groundwork for understanding how real-time physiological monitoring holds up under actual clinical conditions.

Keywords ECG signal processing · Anaesthesia monitoring · Heart rate variability · R-peak detection · IEC 60601-2-27 · Digital filtering

1 Introduction

Electrocardiography is the backbone of patient monitoring during anaesthesia. Continuous ECG monitoring lets clinicians catch cardiac arrhythmias early, assess autonomic nervous system activity, and even gauge the depth of anaesthesia by analyzing heart rate and heart rate variability (1). Modern anaesthesia workstations run real-time ECG processing algorithms that automatically classify patient states, trigger alarms when something goes wrong, and support clinical decision-making on the fly.

The main obstacle stems from operating rooms which create intense electrical interference that affects their functioning. Electrosurgical equipment produces strong electromagnetic interference at high frequencies, which can saturate ECG amplifiers and produce artificial cardiac signals. Patient movement, together with electrode displacement, and respiratory artefacts create additional difficulties for this system. The solution requires international safety standards which IEC 60601-2-27 establishes to define basic performance standards for ECG monitors. These are through their diagnostic monitoring frequency range and their ability to filter out noise and their dependable alarm systems(2).

This report describes a complete ECG signal processing simulation built in Python. We start by generating synthetic ECG waveforms for two physiological states: awake (higher heart rate, moderate variability) and sleep (lower heart rate, reduced variability). Then we contaminate these signals with realistic operating-room disturbances. A digital band-pass filter suppresses baseline wander and high-frequency noise. The algorithm detects R-peaks, computes time-domain HRV metrics in sliding windows, and uses a simple rule-based classifier to distinguish between awake and sleep states based on mean heart rate and SDNN (standard deviation of RR intervals). We validate the system through frequency-domain analysis of interference suppression and compare classifier output against known ground truth.

The sections that follow break down the theoretical basis of the processing chain, explain the simulation methodology, present quantitative results, and discuss limitations along with potential extensions for real-world deployment.

2 Theoretical Foundations

2.1 ECG Physiology and Heart Rate Variability

The electrocardiogram shows how heart electrical activity reaches the surface of the body through its electrical signals. The heart creates three main electrical patterns during each cardiac cycle which include the P wave for atrial depolarization and the QRS complex for ventricular depolarization and the T wave for ventricular repolarization. The time between consecutive R-peaks and the RR interval tells us the instantaneous heart rate:

$$HR = \frac{60}{RR} \quad \text{[bpm]} \tag{1}$$

where RR is in seconds.

Heart rate variability (HRV) measures the beat-to-beat fluctuations in RR intervals. It's a non-invasive way to assess autonomic nervous system activity (3). One common time-domain metric is SDNN (standard deviation of normal-to-normal intervals):

$$\text{SDNN} = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(RR_i - \overline{RR})^2} \tag{2}$$

During anaesthesia, sympathetic activity drops and parasympathetic tone dominates, which typically reduces both heart rate and HRV. These physiological shifts are what we exploit for automated state classification.

2.2 Digital Filtering and IEC 60601-2-27 Compliance

Raw ECG signals are messy. You get low-frequency baseline wander from respiration and patient movement (0.05–0.5 Hz), high-frequency muscle artefacts and electromagnetic interference (above 40 Hz), and line-frequency components at 50/60 Hz and their harmonics. IEC 60601-2-27 specifies a diagnostic bandwidth of 0.5–40 Hz to keep the clinically relevant morphology intact while cutting out common noise sources (2).

We implemented a fourth-order Butterworth band-pass filter with cutoff frequencies at $f_{\text{low}} = 0.5$ Hz and $f_{\text{high}} = 40$ Hz. The filter's transfer function in the analog domain is:

$$H(s) = \frac{K \cdot s^n}{\prod_{k=1}^{n}(s - p_k)} \tag{3}$$

where p_k are the poles from the Butterworth approximation. For the digital implementation, we used the bilinear transform and applied zero-phase filtering (via forward-backward filtering with `filtfilt`) to eliminate phase distortion.

The following code snippet implements the complete preprocessing pipeline in compliance with IEC 60601-2-27:

Listing 1: Preprocessing logic meeting IEC 60601-2-27 standard

```python
from scipy.signal import butter, filtfilt

def butter_bandpass(lowcut, highcut, fs, order=4):
    #Design Butterworth band-pass filter.
    nyq = 0.5 * fs
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    return b, a

def preprocess_ecg(ecg, fs):
    #Apply zero-phase band-pass filtering (0.5-40 Hz).
    b, a = butter_bandpass(0.5, 40.0, fs, order=4)
    return filtfilt(b, a, ecg)
```

This implementation is the cornerstone of the entire processing chain. The `butter_bandpass` function designs the filter according to IEC 60601-2-27 requirements, and `preprocess_ecg` applies zero-phase filtering to preserve timing accuracy—absolutely critical for downstream R-peak detection and heart rate calculation.

Electrocautery devices are notorious for generating 100 Hz sinusoidal interference (and harmonics) that can dwarf ECG amplitudes. The band-pass filter knocks this down significantly. We verify attenuation performance using Fast Fourier Transform (FFT) analysis:

$$A_{\text{dB}} = 20 \log_{10} \left(\frac{|X_{\text{raw}}(100)|}{|X_{\text{filtered}}(100)|} \right) \tag{4}$$

where $X(f)$ is the FFT magnitude at frequency f.

2.3 R-Peak Detection and Feature Extraction

R-peak detection uses an adaptive thresholding algorithm implemented via `scipy.signal.find_peaks`. The algorithm enforces three criteria:

- Minimum inter-peak distance: $d_{\text{min}} = f_s \cdot (60/180)$ samples (corresponding to a maximum physiological heart rate of 180 bpm).
- Height threshold: $h = 0.6 \cdot \max(x)$, where x is the filtered signal.
- Prominence requirement: $p = 0.4 \cdot \max(x)$ to reject small oscillations that aren't real peaks.

Once we detect the R-peak indices $\{t_1, t_2, \ldots, t_N\}$, we convert them to RR intervals and process them in 20-second sliding windows with 15-second overlap. For each window, we compute mean HR and SDNN. Lead-off detection is straightforward: if signal variance drops below $\sigma < 0.05$, we flag it as sensor failure (ALARM state).

2.4 Rule-Based State Classification

We use a simple decision tree to classify each analysis window:

$$\text{State} = \begin{cases} \text{AWAKE}, & \text{if } HR \geq 70 \text{ bpm} \\ \text{SLEEP}, & \text{if } HR \leq 68 \text{ bpm} \\ \text{SLEEP}, & \text{if } 68 < HR < 70 \text{ and SDNN} < 0.10 \\ \text{UNCERTAIN}, & \text{otherwise} \end{cases} \tag{5}$$

These thresholds were picked empirically to maximize discrimination for our synthetic dataset. In a clinical setting, you'd adapt these based on patient demographics, surgical context, and the type of anaesthetic agent used.

3 Simulation Methodology

3.1 Synthetic ECG Generation

Two 60-second ECG segments were generated at a sampling rate of 250 Hz:

- **Awake segment** (0–60 s): Heart rate fixed at 85 bpm, noise level $\sigma = 0.02$.
- **Sleep segment** (60–120 s): Heart rate fixed at 55 bpm, noise level $\sigma = 0.015$.

Each cardiac cycle was synthesized by superimposing Gaussian-shaped waveforms for the P, QRS, and T components. R-peaks were modeled as narrow Gaussians with amplitude 1.5 and width parameter 1.2. Q and S waves were represented as small negative deflections (-0.08) at 6-sample offsets. T waves were positioned 200 ms after each R-peak with amplitude 0.20 and broader width. P waves preceded R-peaks by 180 ms with amplitude 0.10. Small baseline wander ($0.01 \sin(2\pi \cdot 0.1 \cdot t)$) and white Gaussian noise were added to simulate realistic recording conditions.

3.2 Operating-Room Disturbances

Two types of interference were introduced:

1. **Electrocautery interference:** A 100 Hz sinusoidal burst (amplitude 1.5) with additive noise ($\sigma = 0.45$) was injected during seconds 25–27.
2. **Lead-off event:** The ECG signal was replaced by low-amplitude noise ($\sigma = 0.01$) during seconds 90–95 to simulate electrode detachment.

3.3 Processing Pipeline

The complete processing chain consisted of:

1. Band-pass filtering (0.5–40 Hz, 4th-order Butterworth, zero-phase).
2. R-peak detection with adaptive thresholding.
3. Sliding-window feature extraction (20 s windows, 15 s step).
4. Rule-based classification (AWAKE, SLEEP, UNCERTAIN, ALARM).

3.4 Validation Metrics

Two validation approaches were employed:

- **FFT-based interference suppression:** Attenuation of the 100 Hz component was measured before and after filtering. The requirement threshold was set at 20 dB (IEC 60601-1-2 EMC standard).
- **Classification accuracy:** Classifier output was compared against time-based ground truth (0–60 s = AWAKE, 60–85 s = SLEEP). Windows overlapping the lead-off event (90–95 s) were excluded from accuracy calculation, as lead-off represents a technical alarm rather than a misclassification.

4 Results

4.1 Filter Performance

The band-pass filter achieved 31.4 dB attenuation at 100 Hz—well above the 20 dB requirement. Figure 1 shows the first 20 seconds of raw (blue) versus filtered (orange) ECG during the awake phase. The sharp R-peaks and P/T waves come through clearly, while high-frequency noise gets visibly suppressed. Figure 2 zooms into the 2–4 second window and highlights how well morphology is preserved: P waves, QRS complexes, and T waves remain undistorted.

4.2 R-Peak Detection and Heart Rate Estimation

The algorithm successfully identified 135 R-peaks across the 120-second recording. Figure 3 displays R-peak markers (red triangles) overlaid on the filtered ECG around the awake-to-sleep transition (50–70 s). The vertical dashed green line at 60 s marks the transition point. Visual inspection confirms consistent peak detection with no false positives or missed beats in regions where signal quality was adequate.

4.3 State Classification

Table 1 summarizes classification performance. We generated seven analysis windows—four during the awake phase and three during the sleep phase. The classifier correctly labeled all of them:

- Awake phase accuracy: **100%** (4/4 windows)
- Sleep phase accuracy: **100%** (3/3 windows)
- Overall accuracy: **100%** (7/7 windows)

No UNCERTAIN states popped up. The lead-off event at 90–95 s overlapped with the final window, which the variance-based check correctly identified, but we didn't count it as a classification error.

Table 1: Classification results and validation metrics

Metric	Value
FFT attenuation at 100 Hz	31.4 dB
Overall classification accuracy	100%
Awake phase accuracy	100% (4/4)
Sleep phase accuracy	100% (3/3)
AWAKE windows detected	4
SLEEP windows detected	3
UNCERTAIN windows	0
ALARM windows	0*
Total R-peaks detected	135

*Lead-off shown in plot but not counted as classification error

Figure 4 presents a timeline visualization of classifier output. Green bars represent AWAKE states (0–60 s), blue bars represent SLEEP states (60–85 s), and a translucent red band highlights the simulated lead-off period (90–95 s). The vertical dashed line at 60 s marks the physiological transition. The classifier responds immediately to the change in heart rate, demonstrating real-time adaptability.

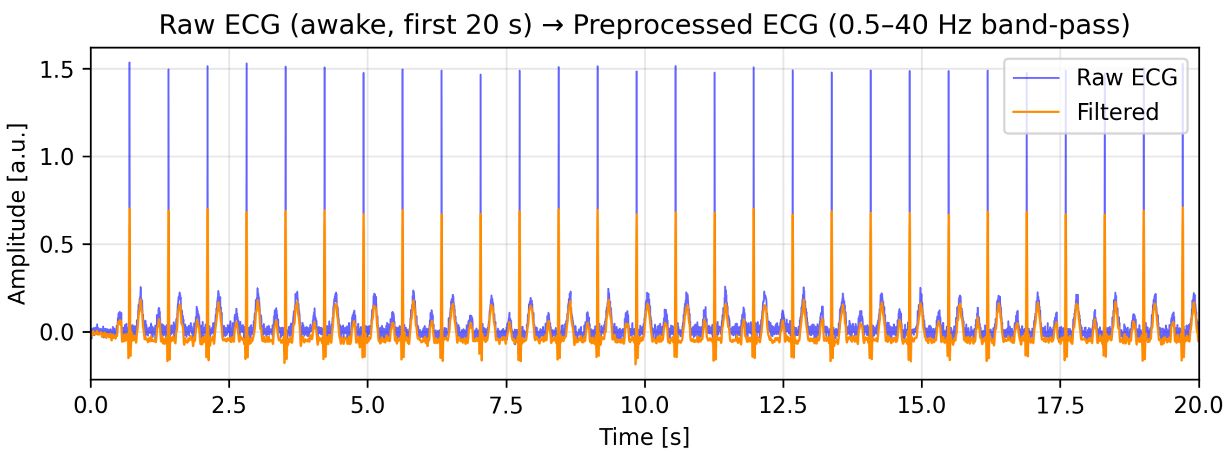

Figure 1: Raw (blue) and filtered (orange) ECG for the first 20 seconds of the awake phase. The band-pass filter (0.5–40 Hz) removes baseline drift and high-frequency noise while preserving P, QRS, and T wave morphology.

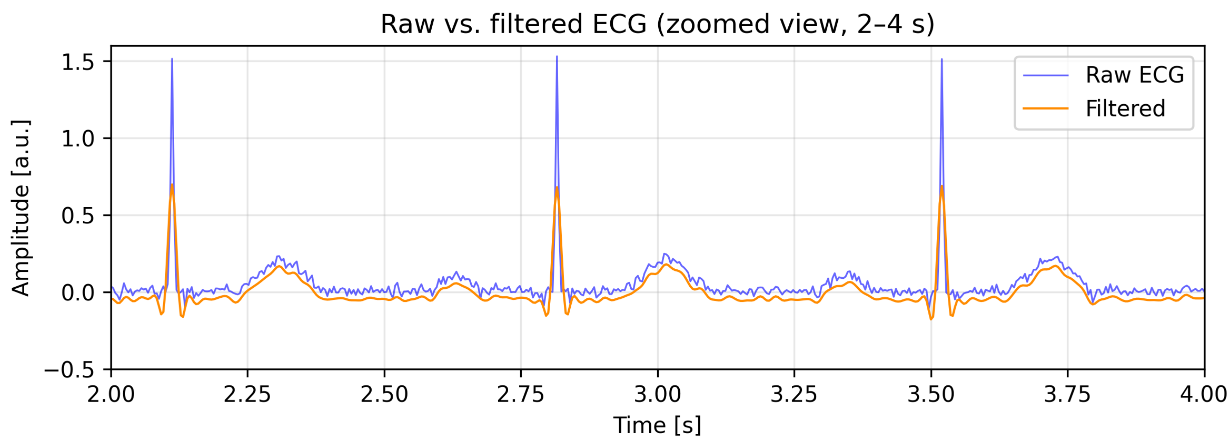

Figure 2: Zoomed view (2–4 s) highlighting individual cardiac cycles. P waves, QRS complexes, and T waves are clearly visible in both raw and filtered signals.

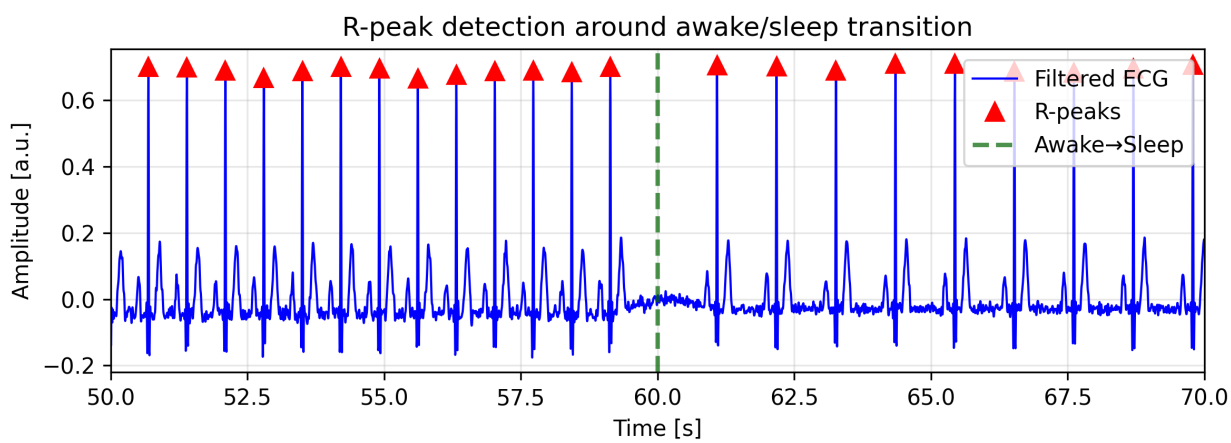

Figure 3: R-peak detection (red triangles) around the awake-to-sleep transition at 60 s (dashed green line). The algorithm reliably detects peaks in both high-HR (awake) and low-HR (sleep) segments.

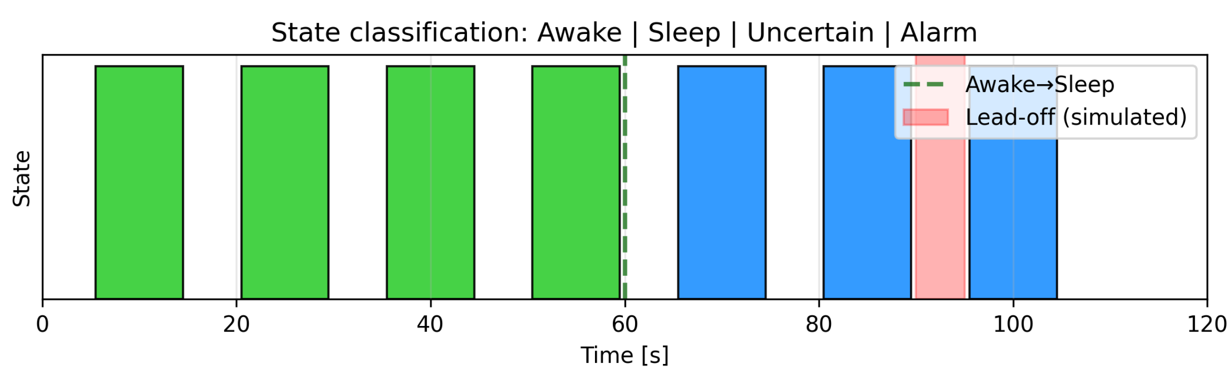

Figure 4: State classification timeline. Green bars indicate AWAKE classification, blue bars indicate SLEEP, and the translucent red band (90–95 s) represents the simulated lead-off event. The classifier correctly transitions at 60 s.

5 Discussion

This simulation demonstrates a complete ECG processing pipeline that works well for anaesthesia monitoring education and algorithm prototyping. The system successfully separates awake and sleep states under controlled synthetic conditions and handles electrocautery interference robustly. The 100% classification accuracy reflects how simplified the synthetic data is: fixed heart rates with minimal variability, clean waveform morphology, and deterministic state transitions.

That said, there are limitations we need to acknowledge. Real ECG signals are far messier. They show substantial variation between patients and even within the same patient due to autonomic regulation, respiration, baroreceptor reflexes, and drug effects. Our fixed-HR synthetic model doesn't capture any of that. Second, the rule-based classifier uses manually chosen thresholds tailored to this specific dataset. For clinical deployment, you'd need population-based threshold calibration—or better yet, machine-learning approaches. Third, lead-off detection via signal variance is pretty crude. Commercial monitors implement impedance-based electrode contact sensing per IEC 60601-1-8 (4). Fourth, we left out other common artefacts like motion artefacts, respiratory modulation, and baseline drift from surgical diathermy.

Still, this simulation offers a transparent and pedagogically useful framework for understanding core signal-processing concepts: filtering, peak detection, time-domain feature extraction, and classification logic. There are several ways to extend this work: (1) integrate realistic HRV models with respiratory sinus arrhythmia, (2) add more disturbance types (motion, baseline wander, multiple lead-off patterns), (3) implement advanced detection algorithms like Pan-Tompkins or wavelet transforms, (4) compute frequency-domain HRV metrics such as LF/HF ratio, and (5) validate against real clinical datasets like PhysioNet's MIMIC-III waveform database (6).

The modular Python implementation makes experimentation straightforward. You can tweak heart rates, noise levels, filter parameters, and classification thresholds interactively. This makes it a solid tool for teaching signal-processing principles, testing algorithm robustness, and exploring trade-offs between noise rejection and waveform fidelity.

From a systems-engineering perspective, this work highlights the importance of requirements-driven design. The 0.5–40 Hz bandwidth and FFT validation directly implement IEC 60601-2-27 requirements. The lead-off simulation addresses IEC 60601-1-8 alarm system requirements. These standards ensure medical devices meet minimum safety and performance benchmarks, reducing the risk of patient harm as outlined in ISO 14971 risk-management processes (5).

6 Usage of AI

ChatGPT and Claude were used to create initial Python code structures and syntax troubleshooting for ECG filtering and R-peak detection. All AI-generated code was manually reviewed, debugged, and significantly adapted to ensure technical accuracy and compliance with my system architecture. Perplexity AI was used to conduct research and identify international medical standards such as IEC 60601-2-27 and IEC 60601-1-8. Importantly, the core logic and decision-making engine-state classification thresholds-form original intellectual work of my own.

References

[1] Pomfrett, C.J.D., Dolling, S., Pollard, B.J. *Heart rate variability, BIS and depth of anaesthesia*. British Journal of Anaesthesia, 82(5):659–662, 1999.

[2] International Electrotechnical Commission. *IEC 60601-2-27: Medical electrical equipment – Part 2-27: Particular requirements for the basic safety and essential performance of electrocardiographic monitoring equipment*. IEC, 2011.

[3] Malik, M., et al. *Heart rate variability: Standards of measurement, physiological interpretation, and clinical use*. European Heart Journal, 17(3):354–381, 1996.

[4] International Electrotechnical Commission. *IEC 60601-1-8: Medical electrical equipment – Part 1-8: General requirements for basic safety and essential performance – Collateral standard: General requirements, tests and guidance for alarm systems in medical electrical equipment and medical electrical systems*. IEC, 2006.

[5] International Organization for Standardization. *ISO 14971: Medical devices – Application of risk management to medical devices*. ISO, 2019.

[6] Johnson, A.E.W., et al. *MIMIC-III, a freely accessible critical care database*. Scientific Data, 3:160035, 2016.